P1 SO: el shell explicado (práctica completa)

Oct 7, 2026 · @Julian

> **Versión completa.** Incluye los 4 grupos, el código ya simplificado (`atoi` en vez de `strtol` , sin `malloc`) y los formatos y mensajes adaptados al shell de referencia y al código de ayuda de los profesores.

Qué es el programa

Nuestro `p1.c` es un **shell**: un programa que pide órdenes, las entiende y las ejecuta, como la terminal de Linux pero hecho por nosotros. Tiene los 21 comandos que pide el enunciado, en 4 grupos: 7 de información (grupo 1, con `exit` y `bye`), 7 para trabajar con ficheros abiertos (grupo 2), 4 para crear y borrar (grupo 3) y 2 para listar (grupo 4).

Funciona siempre igual, en bucle:

1. Escribe el prompt `->` y espera.
2. Lee la línea que escribe el usuario.
3. La parte en palabras: la primera es el comando y las demás sus opciones.
4. Busca ese comando en una tabla y ejecuta su función.
5. Vuelve al paso 1.

Además lleva una **lista de los ficheros que tiene abiertos**, que es la parte central de la práctica. Todo está en un solo fichero, `p1.c` , y se compila con `gcc -Wall p1.c -o p1` .

Lo que hay que saber antes de leer el código

Con estas 12 ideas se entiende todo el programa. Si una parte del código no se entiende, casi seguro que es por una de estas.

1. Un texto en C acaba en `\0` . C no guarda cuánto mide un texto: lo escribe letra a letra y pone un carácter especial, `\0` , al final. Todas las funciones de textos (`printf` con `%s` , `strcmp` , `strcpy`) leen hasta encontrar ese `\0` . Por eso, cuando leemos bytes de un fichero (`readstr`), tenemos que poner el `\0` nosotros, y un texto vacío es el que empieza directamente por `\0` .

2. Un puntero es una dirección de memoria. No guarda el dato, guarda *dónde está*. `char *` es "la dirección donde empieza un texto". Un `&` delante de una variable da su dirección: se usa para que una función pueda escribir en nuestra variable (`uname(&info)` ,

`lstat(nombre, &datos)`). `NULL` es un puntero que no apunta a nada, y lo usamos para marcar "aquí se acaba".

3. Un `struct` agrupa datos. Es como una ficha con varios campos. `struct FICHERO` tiene `nombre` y `modo`, y se accede a cada campo con un punto: `abiertos[3].nombre`. Un array de structs es una tabla: cada posición es una fila.

4. Llamadas al sistema. Hay cosas que nuestro programa no puede hacer solo: abrir un fichero, saber su pid, cambiar de directorio. Se las pide al sistema operativo con funciones como `open`, `close`, `getpid` o `chdir`. Casi todas devuelven **-1 si fallan**, y por eso siempre comprobamos lo que devuelven.

5. `errno` y `perror`. Cuando una llamada al sistema falla, el sistema guarda el motivo en una variable llamada `errno`. `perror("mi mensaje")` imprime nuestro mensaje seguido de ese motivo ya traducido, por ejemplo: `No es posible abrir el fichero: No such file or directory`. Así no tenemos que prever cada error.

6. Descriptor de fichero. Cuando abrimos un fichero, el sistema nos da un número pequeño (3, 4, 5...) y desde ese momento se usa el número en vez del nombre. Todo proceso nace con el 0 (teclado), el 1 (pantalla) y el 2 (errores), heredados del padre. Un fichero abierto tiene un "marcapáginas" (la posición) que avanza al leer o escribir.

7. Los modos son bits. Constantes como `O_CREAT` u `O_APPEND` son números con un solo bit encendido. Con `|` se juntan varias en un número, y con `&` se comprueba si una está encendida. Las tres de lectura/escritura (`O_RDONLY`, `O_WRONLY`, `O_RDWR`) son la excepción: se miran con `& O_ACCMODE`.

8. Memoria: en este programa no hay `malloc`. `malloc(n)` pide memoria mientras el programa corre y `free` la devuelve; si se olvida el `free`, `valgrind` lo marca como fuga. Nosotros no lo necesitamos: la lista de abiertos es un array fijo y `readstr` usa un buffer fijo de `MAX_TEXT`. Por eso es imposible que haya fugas de memoria.

9. Las constantes en mayúsculas (`MAX_TEXT`, `O_CREAT`, `SEEK_SET`, `F_GETFL`) son números con nombre. Las nuestras las definimos con `#define`; las del sistema vienen en los `#include`. Lo que significa cada una está en el manual: `man 2 open`, `man 2 lseek`, `man strftime`...

10. `lstat` y el `struct stat`. `lstat(nombre, &datos)` rellena una ficha con todo lo que el sistema sabe de un fichero: tipo y permisos (`st_mode`), tamaño (`st_size`), dueño (`st_uid`), grupo (`st_gid`), número de enlaces (`st_nlink`), inodo (`st_ino`) y fechas (`st_mtime` modificación, `st_atime` último acceso). `S_ISDIR(datos.st_mode)` dice si es un directorio y `S_ISLNK`, si es un enlace. Usamos `lstat` y no `stat` porque, si el nombre es un enlace simbólico, `stat` mira a dónde apunta y `lstat` mira el enlace en sí.

11. Recorrer un directorio. `opendir(nombre)` lo abre, `readdir` va dando lo que hay dentro de uno en uno (y `NULL` al acabar) y `closedir` lo cierra. Cada cosa leída es un `struct dirent`, y su nombre está en `entrada->d_name`. Todo directorio contiene `.` (él mismo) y `..` (el de arriba), que casi siempre hay que saltarse.

12. Recursividad. Una función recursiva es una que se llama a sí misma. Sirve para los directorios porque dentro de una carpeta puede haber otra carpeta, y dentro otra, sin saber cuántas: la función hace lo mismo con cada subcarpeta que encuentra. Siempre necesita un caso en el que deja de llamarse (aquí, cuando no quedan subcarpetas), o no acabaría nunca.

Cómo está ordenado p1.c

C lee el fichero de arriba abajo y solo conoce lo que ya ha visto. Por eso el orden importa: primero lo que se usa, después quien lo usa.

Orden	Qué hay	Para qué
1	Comentario con los autores	Lo pide el enunciado
2	`#include`, agrupados por grupo de comandos	Traen las funciones del sistema (`printf`, `open`, `lstat`, `opendir`, `getpwuid`…)
3	`#define`	Tamaños máximos: `MAX_TEXT` 2048, `MAX_NOMBRE` 1024, `MAX_ABIERTOS` 1024
4	Prototipos de las `Cmd_`	Avisan de que esas funciones existen, porque la tabla las nombra antes de que estén escritas
5	`struct COMANDO` y la tabla `comandos[]`	Qué comandos hay, qué función ejecuta cada uno y su ayuda
6	`struct FICHERO` y el array `abiertos[]`	La lista de ficheros abiertos
7	`TrocearCadena`	Parte la línea en palabras (la da el enunciado)
8	`InicializarAbiertos`, `ModoTexto`, `LeerDescriptor`, `MostrarDirectorioActual`	Funciones de ayuda: no son comandos
9	`Cmd_` de los grupos 1 y 2	Un comando cada una

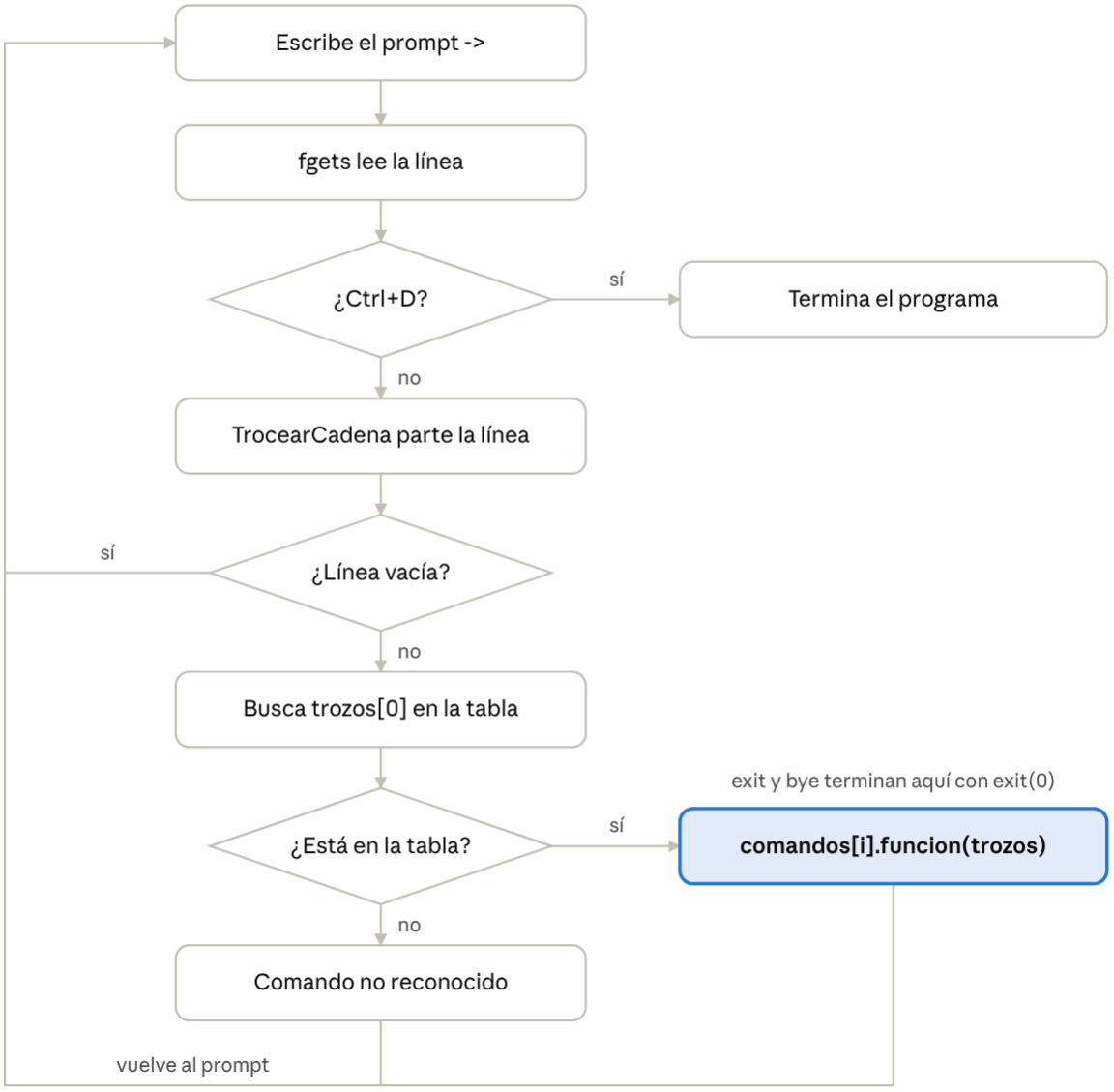
Orden	Qué hay	Para qué
10	Grupo 3: `Cmd_makefile`, `Cmd_makedir`, `BorrarUno`, `Cmd_delete`, `BorrarArbol`, `Cmd_deltree`	Crear y borrar
11	Grupo 4: `struct OPCIONES`, `LeerOpciones`, `LetraTF` y `ConvierteModo` (de los profesores), `InfoFichero`, `Cmd_listfile`, `RecorrerSubdirectorios`, `ListarDirectorio`, `Cmd_list`	Listar
12	`main`	Arranca el programa y tiene el bucle del shell

Todas las funciones de comandos tienen la misma forma, `void Cmd_algo(char *trozos[])`: reciben la línea troceada y no devuelven nada. En `trozos[0]` está el nombre del comando y desde `trozos[1]` sus opciones.

El main y TrocearCadena

El `main` es el "motor" del shell: prepara la lista de abiertos y luego da vueltas sin parar pidiendo órdenes.

El shell repite el mismo ciclo hasta exit, bye o Ctrl+D



el bucle del main · 3 decisiones

Cada orden recorre este camino de arriba abajo y vuelve al prompt; solo Ctrl+D, exit y bye lo cortan.

Variables del main.

- `cadena[MAX_TEXT]` : aquí se guarda la línea que escribe el usuario, de hasta 2047 letras.
- `trozos[MAX_TEXT/2 + 1]` : la lista de palabras. Son punteros que apuntan dentro de `cadena` . Mide 1025 porque, en el peor caso (`a a a a...`), hay 1024 palabras, y hace falta un hueco más para el `NULL` del final. Si fuera más pequeño, una línea con muchas palabras escribiría fuera del array: segmentation fault.
- `i` : para recorrer la tabla de comandos.

Antes del bucle, `InicializarAbiertos()` mete en la lista el 0, el 1 y el 2.

El bucle `while (1)` es infinito a propósito. En cada vuelta:

1. `printf("-> ")` y `fflush(stdout)`. El `fflush` obliga a que el prompt salga ya: sin él, como no hay salto de línea, C podría guardarlo y no enseñarlo.
2. `fgets(cadena, MAX_TEXT, stdin)` lee una línea entera del teclado. Si devuelve `NULL`, el usuario pulsó **Ctrl+D** (fin de fichero) y salimos con `break`.
3. `TrocearCadena(cadena, trozos)` parte la línea. Si devuelve 0, la línea estaba vacía (solo Enter) y con `continue` volvemos al principio.
4. Un `for` busca `trozos[0]` en la tabla de comandos y llama a su función (está explicado en la sección de la tabla).

Del bucle solo se sale de dos formas: con Ctrl+D (el `break`) o con `exit` / `bye`, que terminan el programa directamente.

TrocearCadena (la da el enunciado). Usa `strtok`, que busca espacios, tabuladores y saltos de línea en `cadena`, los cambia por `\0` y devuelve dónde empieza cada palabra. No copia nada: las palabras siguen dentro de `cadena`, y `trozos[i]` solo apunta a ellas. Con `pid -p`, la memoria queda así:

```
cadena:  p i d \0 - p \0
         ^          ^
     trozos[0]  trozos[1]      trozos[2] = NULL
```

La primera llamada es `strtok(cadena, ...)` y las siguientes `strtok(NULL, ...)`, que significa "sigue por donde ibas". Cuando no quedan palabras devuelve `NULL`, y ese `NULL` queda guardado al final de `trozos`. Gracias a eso, `trozos[1] == NULL` significa "el comando no trae opciones".

La tabla de comandos

La tabla `comandos[]` es la "agenda" del shell: para cada comando dice cómo se llama, qué función lo hace y qué ayuda mostrar. Es la idea de la versión 3 de los ejemplos de los profesores.

Cada fila es un `struct COMANDO` con tres campos:

- `nombre`: el texto que escribe el usuario, por ejemplo `"date"`.
- `funcion`: la función que lo ejecuta, por ejemplo `Cmd_date`. Se declara `void (*funcion)(char **)`, que se lee "un puntero a una función que recibe `char **` y no devuelve nada".
- `texto`: lo que muestra `help date`.

La última fila es `{NULL, NULL, NULL}` y marca el final, igual que el `NULL` de `trozos`.

Puntero a función, como para tontos. En C, el nombre de una función sin paréntesis (`Cmd_date`) es la dirección donde está su código, y se puede guardar en una variable como cualquier otra dirección. Con paréntesis (`Cmd_date(trozos)`) se ejecuta. Así que la tabla guarda "dónde está el código de cada comando", y luego se llama a través de ella: `comandos[i].funcion(trozos)`.

Por qué todas las `Cmd_` tienen la misma forma. Para caber en la misma columna, todas tienen que recibir `char *trozos[]` y devolver `void`, aunque no usen los argumentos. Cuando una no los usa (`sysinfo`, `exit`, `listopen`) ponemos `(void) trozos;`: es un *cast* a "nada" que no hace nada, pero le dice al compilador que no lo usamos a propósito, y así no avisa con `-Wextra`.

Cómo la usa el main. Si el usuario escribe `date -d`:

1. `i = 0`: `"authors"` no es `"date"`, sigue.
2. `i = 1`: `"pid"` no es `"date"`, sigue.
3. `i = 2`: `"date"` coincide, ejecuta `comandos[2].funcion(trozos)`, que es `Cmd_date(trozos)`, y hace `break`.

Si escribe `patata`, ninguna fila coincide, el `for` llega a la fila `NULL`, y como `comandos[i].nombre == NULL` se imprime "Comando no reconocido". Ojo: el `break` del `for` solo sale del `for` (deja de buscar), no del `while`.

Por qué una tabla y no muchos `if`. Con una cadena de `if / else if`, el `main` crecería con cada comando y el nombre estaría repetido en el `main` y en la ayuda. Con la tabla, añadir un comando es escribir su función, su prototipo arriba y una fila, sin tocar el `main`, y `help` lo conoce solo. No es obligatoria (el enunciado no lo pide), pero es lo que proponen los profesores como mejora.

Los prototipos de arriba (`void Cmd_date (char *trozos[]);`) existen porque la tabla nombra las funciones antes de que estén escritas, y C lee de arriba abajo. El prototipo le dice "existe y tiene esta forma, la escribo luego".

Grupo 1: comandos de información

Son comandos cortos que preguntan algo al sistema y lo enseñan. Casi todos siguen el mismo esquema: miran `trozos[1]` para saber la opción y, según cuál sea, hacen una cosa u otra.

`authors [-l|-n]`. Enseña quiénes hicimos la práctica. Si `trozos[1]` es `NULL` (sin opción) saca nombres y logins; con `strcmp` mira si es `-n` (solo nombres) o `-l` (solo logins). Cualquier otra cosa da un mensaje de opción no válida. `strcmp` devuelve 0 cuando los dos textos son iguales.

`pid [-p]`. Enseña el número del proceso del shell con `getpid()` (`Pid de shell: 517`), o el de su padre, la terminal que lo lanzó, con `getppid()` si viene `-p` (`Pid del padre del shell: 516`). Fijaos en el orden de la condición: `trozos[1] != NULL && strcmp(...)`. Primero se comprueba que exista, porque un `strcmp` con `NULL` rompe el programa, y `&&` no mira la segunda parte si la primera ya es falsa.

`date [-d|-t]`. Saca la fecha y la hora en tres pasos:

1. `time(NULL)` da la hora actual como un número: los segundos desde el 1 de enero de 1970.
2. `localtime(&t)` convierte ese número en una `struct tm` con día, mes, año, hora... ya en hora de España. Si devuelve `NULL`, `perror`.
3. `strftime` escribe esa fecha como texto con el formato que le pidamos: `%d` día, `%m` mes, `%Y` año, `%H` hora, `%M` minutos, `%S` segundos (vienen en `man strftime`, no confundir con los `%` de `printf`).

La fecha se escribe con `"%d/%m/%Y"` (`07/10/2026`) y la hora con `"%H:%M:%S"`. Sin opción imprime **primero la hora y luego la fecha**, como el shell de referencia; con `-d` solo la fecha y con `-t` solo la hora.

`chdir [dir]`. Sin argumento, `getcwd` escribe en `dir` la carpeta donde está el shell y la imprime. Con argumento, `chdir(trozos[1])` intenta cambiar a esa carpeta: si devuelve -1, `perror("Imposible cambiar directorio")` dice el motivo (no existe, no es un directorio, permiso denegado). Si va bien no imprime nada, como `cd`. Tiene que ser un comando del propio shell porque cada proceso tiene su propia carpeta actual: un programa aparte solo cambiaría la suya.

`sysinfo`. Llama a `uname(&info)`, que rellena una `struct utsname` con los datos de la máquina: nombre del equipo (`nodename`), procesador (`machine`), sistema (`sysname`) y versión del kernel (`release` y `version`). Los imprime como el shell de referencia: `vm (x86_64), OS: Linux-6.18...`. El `&` es para que `uname` pueda escribir en nuestra variable.

`help [cmd]`. Recorre la tabla de comandos hasta la fila `NULL`. Sin argumento imprime el nombre de cada fila. Con argumento busca la fila cuyo nombre coincide con `trozos[1]` e imprime `nombre: texto`, por ejemplo `date: [-d|-t] Muestra la fecha...`, como la referencia; después hace `return`. Si el bucle termina sin encontrarla, dice "no encontrado". Como usa la misma tabla que el `main`, siempre conoce todos los comandos.

`exit` y `bye`. Las dos filas apuntan a la misma función, `Cmd_fin`, que llama a `exit(0)`. `exit` termina el programa al momento, esté donde esté, y el 0 significa "acabó bien".

Grupo 2: la lista de ficheros abiertos

El enunciado pide que el shell lleve su propia lista de ficheros abiertos (al menos nombre y descriptor) y que coincida con la del sistema, que se ve con `lsof -p <pid>` en otra terminal.

Cómo es la lista: una estantería con casillas. Es un array global, `struct FICHERO abiertos[MAX_ABIERTOS]`. Cada `struct FICHERO` tiene `nombre` y `modo`. El truco es que **la posición del array es el descriptor**: el fichero con descriptor 3 va en `abiertos[3]`. Si el nombre de una casilla está vacío (empieza por `\0`), ese descriptor está cerrado.

Casilla	nombre	modo
0	entrada estandar	O_RDONLY
1	salida estandar	O_WRONLY
2	error estandar	O_WRONLY
3	prueba.txt	O_RDWR O_CREAT
4	(vacío = cerrado)	

Así se ve la lista tras `open prueba.txt cr rw`. Los modos de 0, 1 y 2 dependen de cómo los abrió la terminal.

Por qué así y no con la `listasimple` de los profesores. El enunciado no dice cómo hacerla, y `listasimple` es solo un ejemplo. Con el array no hay `malloc`, así que no puede haber fugas de memoria; todo está en un fichero y no hace falta Makefile; y buscar un descriptor es directo (`abiertos[df]`), sin recorrer nada. Como es una variable global, C la pone entera a 0 al empezar: todas las casillas vacías.

Funciones de ayuda (no son comandos, no van en la tabla).

- `InicializarAbiertos`: se llama una vez al principio del `main`. Escribe los nombres de 0, 1 y 2, porque el shell los hereda de su padre ya abiertos. Como no los abrimos nosotros, preguntamos su modo al sistema con `fcntl(df, F_GETFL)` ("dime con qué modos está abierto").
- `ModoTexto(modo, texto)`: convierte el número de modo en palabras para `listopen`. Primero `modo & O_ACCMODE` se queda solo con la parte de lectura/escritura y un `switch` elige entre `O_RDONLY`, `O_WRONLY` y `O_RDWR`. Después, con `&`, mira uno a uno si están encendidos `O_CREAT`, `O_EXCL`, `O_APPEND` y `O_TRUNC`, y los añade con `strcat` (que pega un texto al final de otro).

- `LeerDescriptor(texto)`: los comandos reciben el descriptor como texto (`"3"`), no como número. Primero mira a mano que el primer carácter sea una cifra (`texto[0] < '0' || texto[0] > '9'` no vale); luego `atoi` lo pasa a número y se comprueba que no sea mayor que `MAX_ABIERTOS - 1`. Si algo falla, avisa y devuelve -1. Sin esto, `close 99999` escribiría fuera del array (segmentation fault), y `close patata` cerraría el 0, el teclado, porque `atoi("patata")` da 0 (es justo lo que le pasa al shell de referencia).
- `MostrarDirectorioActual`: escribe la carpeta actual con `getcwd`. La usan `makefile`, `makedir`, `delete`, `deltree`, `listfile` y `list` cuando no les damos ningún nombre, igual que el shell de referencia.

Grupo 2: comando a comando

Para entender estos comandos sirve imaginar un fichero abierto como un libro con un **marcapáginas**: el marcapáginas es la posición por donde vas leyendo o escribiendo, y avanza solo cada vez que lees o escribes.

`listopen`. Mira la estantería casilla por casilla (un `for` de 0 a `MAX_ABIERTOS`) y, para cada una con nombre, pasa su modo a palabras con `ModoTexto` y pregunta por dónde va su marcapáginas con `lseek(df, 0, SEEK_CUR)`: moverse 0 desde donde está devuelve la posición actual. Imprime `descriptor: 3, offset: (12)-> prueba.txt O_RDWR O_CREAT`. La terminal (0, 1, 2) a veces no tiene posición: entonces `lseek` da -1 y sale `offset: ( )`, como en la referencia.

`open fich modos`. Pide al sistema que abra un fichero.

1. Sin argumentos, llama a `listopen` y termina.
2. Recorre las palabras desde `trozos[2]` y traduce cada modo: `cr` → `O_CREAT` (créalo si no existe), `ex` → `O_EXCL` (falla si ya existe), `ro` / `wo` / `rw` → solo leer / solo escribir / las dos, `ap` → `O_APPEND` (escribe siempre al final), `tr` → `O_TRUNC` (vácialo al abrir). Las va juntando con `modo |= ...`.
3. `open(trozos[1], modo, 0777)` devuelve el descriptor o -1. El `0777` son los permisos del fichero si se crea (en octal: leer, escribir y ejecutar para todos; el sistema luego quita algunos).
4. Si va bien, copia el nombre a `abiertos[df]` con `strncpy` (que copia como mucho `MAX_NOMBRE - 1` letras, para no desbordar) y guarda el modo.

`close df`. Pide al sistema que cierre un descriptor. Lo comprueba con `LeerDescriptor`, llama a `close(df)` y, si no da -1, deja el nombre vacío en `abiertos[df]`. Si ya estaba cerrado, el sistema se queja y lo mostramos con `perror("Imposible cerrar descriptor")`: `Bad file descriptor`.

`dup df`. Es como hacer una **copia de una llave**: `dup(df)` da un descriptor nuevo (el número libre más bajo) que abre el mismo fichero. Las dos llaves comparten el marcapáginas: si lees 4 letras con una, la otra también está 4 letras más adelante. En la lista se guarda con el nombre `dup 3 (prueba.txt)`, escrito con `snprintf` (como `printf` pero en un texto y sin pasarse del tamaño), y con el mismo modo que el original.

El nombre se monta como en el código de ayuda de los profesores, `sprintf(aux, "dup %d (%s)", ...)`, y se copia a la lista con `strncpy`, igual que en `open`. El modo se pregunta al sistema con `fcntl(nuevo, F_GETFL)`; por eso el `dup` sale `O_RDWR` sin `O_CREAT`: `O_CREAT` solo cuenta al abrir y el sistema no lo guarda.

`lseek df pos ref`. Mueve el marcapáginas. `ref` dice desde dónde se cuenta: `SEEK_SET` desde el principio, `SEEK_CUR` desde donde está ahora y `SEEK_END` desde el final. La posición se pasa a número con `atol` (como `atoi`, pero para `long`) y puede ser negativa: `lseek 3 -3 SEEK_END` lo pone 3 letras antes del final. `lseek` devuelve la nueva posición o -1, y se imprime `Descriptor 3 posicionado en 9`. Si faltan argumentos: `Parametros incorrectos`.

`readstr df cont`. Lee `cont` letras desde el marcapáginas y las enseña.

1. Comprueba el descriptor y que `cont` (pasado con `atoi`) sea positivo; si no, `Parametros incorrectos`.
2. Lee en un buffer fijo, `char buffer[MAX_TEXT]`. Si piden más de lo que cabe, se lee `MAX_TEXT - 1` (el último hueco es para el `\0`). No usa `malloc`, así que no hay nada que liberar.
3. `read(df, buffer, cont)` lee y devuelve cuántos bytes ha leído de verdad (pueden ser menos si el fichero se acaba), o -1 si falla.
4. Pone el `\0` justo después de lo leído e imprime `Leidos 4 bytes del descriptor 4` y, debajo, el texto.

`writestr df texto`. Escribe el texto donde esté el marcapáginas. Como `TrocearCadena` había partido `hola que tal` en tres palabras, las recorre desde `trozos[2]` y escribe cada una con `write(df, palabra, strlen(palabra))`, con un espacio entre medias. Si va bien no imprime nada, igual que el shell de referencia; si `write` devuelve -1, `perror("Imposible escribir")`.

Todo junto, en una prueba.

```
-> open prueba.txt cr rw tr     → Anadida entrada 3 a la tabla ficheros
abiertos
-> writestr 3 hola que tal      → (no dice nada; el marcapáginas queda al
final)
-> dup 3                        → entrada 4 (copia de la llave del 3)
-> lseek 3 0 SEEK_SET           → Descriptor 3 posicionado en 0 (para el 3 y
el 4)
-> readstr 4 4                  → Leidos 4 bytes del descriptor 4: hola
-> readstr 3 100                → Leidos 8 bytes: " que tal" (sigue donde lo
dejó el 4)
-> close 4
-> listopen                     → 0, 1, 2 y 3, cada uno con su offset
```

Grupo 3: crear y borrar

Cuatro comandos para crear y borrar ficheros y carpetas. Sin ningún nombre, los cuatro enseñan la carpeta actual con `MostrarDirectorioActual`, como el shell de referencia.

`makefile nombre`. Crea un fichero vacío con `open(nombre, O_CREAT | O_EXCL | O_WRONLY, 0777)`. `O_CREAT` lo crea y `O_EXCL` hace que falle si ya existe, para no machacar uno que ya estaba. Como solo queríamos crearlo, se cierra al momento con `close` y **no** se mete en la lista de abiertos. Si falla: `Imposible crear fichero`.

`makedir nombre`. Crea una carpeta con `mkdir(nombre, 0755)`. El `0755` son los permisos en octal: el dueño puede todo y los demás, leer y entrar. Si ya existe o no hay permiso: `Imposible crear directorio`.

`BorrarUno` y `delete n1 n2...`. `BorrarUno` mira con `lstat` qué es el nombre. Si `S_ISDIR` dice que es una carpeta, la borra con `rmdir`, que **solo funciona si está vacía**; si no, con `unlink`, que borra ficheros y enlaces. `delete` llama a `BorrarUno` con cada nombre desde `trozos[1]`; si uno falla, avisa (`Imposible borrar noexiste: No such file or directory`) y sigue con el siguiente. El motivo se escribe con `strerror(errno)`, que da el mismo texto que `perror` pero para meterlo en un `printf`.

`BorrarArbol` y `deltree n1 n2...` (recursivo). Es como `delete`, pero vacía las carpetas antes de borrarlas:

1. Si el nombre no es una carpeta, se borra con `BorrarUno` y ya está.
2. Si lo es, la abre con `opendir` y la recorre con `readdir`.
3. Se salta `.` y `..`. **Esto es clave:** si no se saltara `..`, entraría en la carpeta de arriba y lo borraría todo.

4. Para cada cosa de dentro, monta la ruta `carpeta/cosa` con `snprintf` y **se llama a sí misma** con esa ruta. Si dentro hay otra carpeta, hace lo mismo con ella, y así hasta el fondo.
5. Cierra con `closedir` y, como la carpeta ya está vacía, la borra con `BorrarUno` (`rmdir`).

Por qué `lstat` y no `stat`. Si el nombre es un enlace simbólico a una carpeta, `stat` diría "es una carpeta" y `deltree` vaciaría la carpeta de destino, que puede estar en cualquier sitio. Con `lstat`, el enlace es un enlace: se borra solo el enlace con `unlink`.

```
-> makedir d
-> makefile d/x
-> makedir d/sub
-> makefile d/sub/y
-> delete d                → Imposible borrar d: Directory not empty
-> deltree d               → borra d con todo lo que tiene dentro
```

Grupo 4: listar

`listfile` habla de cada nombre en sí; `list` hace lo mismo, pero si el nombre es una carpeta enseña **lo que hay dentro**. Los dos comparten casi todo el código.

Las opciones (`struct OPCIONES` y `LeerOpciones`). Las seis opciones se guardan en una ficha con un campo por opción, a 1 si se ha puesto y a 0 si no: `largo` (`-long`), `link` (`-link`), `acc` (`-acc`), `hid` (`-hid`), `reca` (`-reca`) y `recb` (`-recb`). `LeerOpciones` pone las 6 a 0 y recorre las palabras desde `trozos[1]` mientras sean opciones; la primera que no lo es ya es un nombre, y devuelve su posición. Recibe la ficha como puntero (`struct OPCIONES *op`) para poder rellenar la del que la llama, por eso se escribe `op->largo` y no `op.largo` (es lo mismo que `(*op).largo`).

Los permisos (`LetraTF` y `ConvierteModo`, de los profesores). Están copiadas tal cual del código de ayuda. `LetraTF` mira el tipo en `st_mode` y da su letra: `d` carpeta, `l` enlace, `-` fichero normal, `p` tubería... `ConvierteModo` empieza con `"---------- "` y va poniendo `r`, `w` o `x` donde el bit está encendido, en tres grupos (dueño, grupo y otros), más `s` y `t` para los permisos especiales. Resultado: `-rw-r--r--` o `drwxr-xr-x`, como en `ls -l`.

`InfoFichero(ruta, nombre, op)`: la línea de un fichero. Recibe la ruta para encontrarlo y el nombre que se escribe en pantalla.

1. `lstat(ruta, &datos)`. Si falla: `****error al acceder a noexiste:No such file or directory`, como la referencia.
2. **Sin `-long`:** tamaño y nombre, `5 f.txt`.

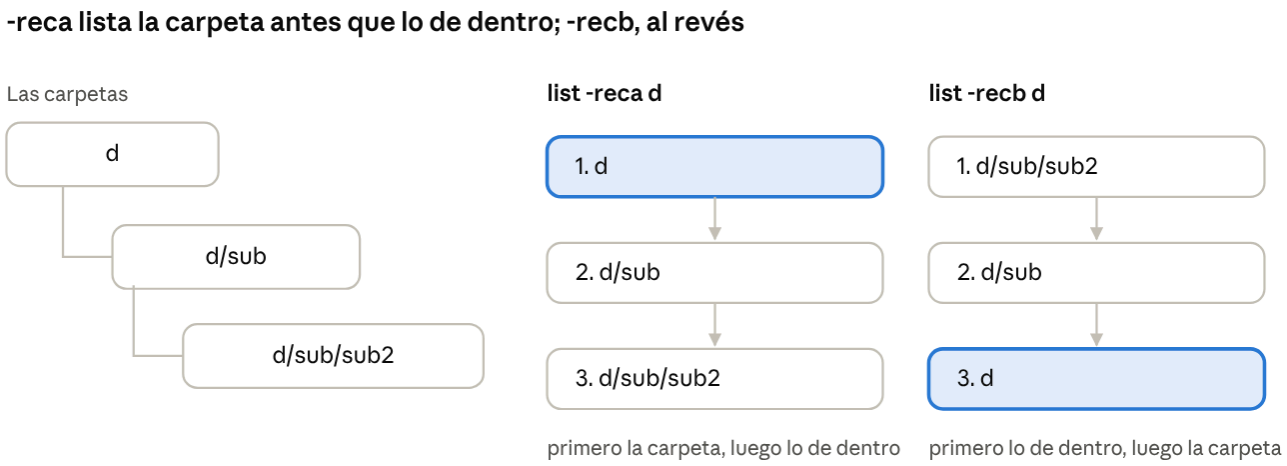
3. **Con `-long`:** fecha (modificación, o último acceso con `-acc`) escrita con `strftime` como `2026/10/07-22:30`, número de enlaces, inodo entre paréntesis, dueño, grupo, permisos, tamaño y nombre, todo en una línea.
4. **Con `-link` y si es un enlace:** añade `-> destino`. `readlink` da a dónde apunta, pero **no pone el `\0`**, así que hay que ponerlo a mano.

El usuario "sin dueño" (la trampa del enunciado). `getpwuid(st_uid)` y `getgrgid(st_gid)` pasan el número de usuario y de grupo a su nombre. Si el usuario ya no existe, devuelven `NULL`, y usar `pw->pw_name` daría **segmentation fault**. Por eso, si es `NULL`, escribimos `??????`, como la referencia.

`listfile [-long] [-link] [-acc] n1 n2...`. Lee las opciones y llama a `InfoFichero` con cada nombre. Si es una carpeta, da la información de la carpeta en sí, no de lo de dentro.

`list [-reca] [-recb] [-hid] [...] n1 n2...`. Con cada nombre hace `lstat`: si no es una carpeta, igual que `listfile`; si lo es, llama a `ListarDirectorio`.

- `ListarDirectorio(dir, op)` escribe la cabecera `**********d` y recorre la carpeta con `opendir` / `readdir`, llamando a `InfoFichero` con cada cosa. Los nombres que empiezan por `.` (ocultos, y también `.` y `..`) solo salen con `-hid`.
- `RecorrerSubdirectorios(dir, op)` busca las subcarpetas de `dir` (sin `.` ni `..`, y sin ocultas salvo con `-hid`) y llama a `ListarDirectorio` con cada una. **Aquí está la recursividad:** las dos funciones se llaman la una a la otra, por eso una necesita el prototipo arriba.
- `-recb` llama a `RecorrerSubdirectorios` **antes** de listar la carpeta, y `-reca`, **después**. Es la única diferencia entre las dos.
- Como se mira con `lstat`, un enlace a una carpeta no cuenta como carpeta y no se entra en él. Así no hay bucles infinitos aunque un enlace apunte hacia arriba.



orden de las cabeceras con -reca y -recb · 3 carpetas

Las dos opciones listan las mismas carpetas; solo cambia el orden: la carpeta de arriba sale la primera con `-reca` y la última con `-recb`.

Adaptación al shell de referencia y al código de los profesores

Ejecutamos el shell de referencia (`0LINUX/shell.out`) con las mismas órdenes que el nuestro y ajustamos los formatos y mensajes para que salgan igual. Del código de ayuda copiamos `LetraTF` y `ConvierteModo` tal cual, y seguimos su forma de hacer `dup`.

Comando	Cómo sale ahora (igual que la referencia)
`pid` / `pid -p`	`Pid de shell: 517` / `Pid del padre del shell: 516`
`date`	Hora y debajo fecha: `22:28:27` y `07/10/2026`
`sysinfo`	`vm (x86_64), OS: Linux-6.18...`
`help date`	date: [-d o -t] Muestra la fecha y/o la hora actual (nombre, dos puntos y la ayuda)
`chdir noexiste`	`Imposible cambiar directorio: No such file or directory`
`open`	`Anadida entrada 3 a la tabla ficheros abiertos` / `Imposible abrir fichero: ...`
`close` , `dup`	`Imposible cerrar descriptor: ...` / `Imposible duplicar descriptor: ...`
`listopen`	`descriptor: 3, offset: (12)-> p.txt O_RDWR O_CREAT`
`lseek`	`Descriptor 3 posicionado en 0`
`readstr`	`Leidos 4 bytes del descriptor 4` y el texto debajo
`lseek` , `readstr` , `writestr` sin argumentos	`Parametros incorrectos`
`makefile` , `makedir` , `delete` , `deltree` , `listfile` , `list` sin nombres	Enseñan la carpeta actual
`delete` , `deltree`	`Imposible borrar noexiste: No such file or directory`

Comando	Cómo sale ahora (igual que la referencia)
`listfile`, `list`	Tamaño en 8 columnas, `****error al acceder a noexiste:...` y `??????` si no hay dueño

Diferencias que dejamos a propósito (por si preguntan):

- `authors`: la referencia pone "Nombre del autor1"; nosotros, nuestros nombres reales.
- `listopen`: la referencia enseña del 0 al 19, también los "no usado"; el nuestro solo los abiertos, que es más claro y cumple el enunciado.
- `dup`: la referencia lo llama `dupplicado de 3 (p.txt)` (con errata); el código de ayuda usa `dup 3 (p.txt)`, que es el que tenemos.
- `close patata`: la referencia usa `atoi` sin comprobar y **cierra el teclado** (el 0). El nuestro comprueba que empiece por una cifra y dice `Descriptor no valido`.

Preguntas típicas de la defensa

Pregunta	Respuesta corta
¿Por qué `trozos` es un array de punteros?	`strtok` no copia las palabras: las deja en `cadena` y cada `trozos[i]` apunta a una. No se reserva memoria.
¿Por qué `trozos` mide `MAX_TEXT/2 + 1`?	El peor caso (`a a a...`) da 1024 palabras, más 1 para el `NULL` final. Más pequeño = posible segmentation fault.
¿Para qué sirve el `fflush(stdout)`?	Para que el prompt salga ya, aunque no lleve salto de línea.
¿Qué es un puntero a función?	Una variable que guarda dónde está el código de una función, para llamarla desde la tabla.
¿Por qué todas las `Cmd_` reciben `char *trozos[]`?	Para caber en la misma columna de la tabla y llamarlas igual. Si no lo usan, `(void) trozos;`.
¿Cómo sabe el main que un comando no existe?	El `for` llega a la fila `{NULL, NULL, NULL}` sin encontrarlo.

Pregunta	Respuesta corta
¿Por qué `chdir` tiene que ser interno?	Cada proceso tiene su propia carpeta actual; un programa aparte solo cambiaría la suya.
¿Qué es un descriptor?	El número que da el sistema al abrir un fichero: su posición en la tabla de abiertos del proceso.
¿Por qué el primer `open` da 3?	0, 1 y 2 ya están abiertos (heredados del padre) y `open` da el libre más bajo.
¿Por qué la lista empieza con 0, 1 y 2?	Para coincidir con `lsof -p`: el proceso los hereda abiertos.
¿Por qué un array indexado por descriptor?	Sin `malloc` no hay fugas, todo en un fichero y `abiertos[df]` es acceso directo.
¿Cómo se juntan y se comprueban los modos?	Son bits: el OR (barra vertical) los junta y el AND (&) comprueba uno. Lectura/escritura con & O_ACCMODE.
¿Qué diferencia hay entre `dup` y abrir dos veces?	Con `dup` los dos descriptores comparten la posición; abriendo dos veces, cada uno tiene la suya.
¿Cómo sabéis por dónde va un fichero en `listopen`?	`lseek(df, 0, SEEK_CUR)`: moverse 0 desde donde está devuelve la posición actual.
¿Por qué `close patata` no cierra el 0?	`LeerDescriptor` comprueba que empiece por una cifra antes de usar `atoi`, que con texto devolvería 0.
¿Por qué el buffer de `readstr` deja un hueco?	Para el `\0`, que hace falta para imprimirlo como texto. Si piden más, se lee `MAX_TEXT - 1`.
¿Cómo evitáis fugas?	No hay ningún `malloc` en el programa: todo son arrays fijos. Se comprueba con `valgrind`.
¿Para qué sirve `perror`? ¿Y `strerror(errno)`?	`perror` muestra nuestro mensaje y el motivo del error de `errno`; `strerror(errno)` da ese motivo como texto para meterlo en un `printf`.
¿Por qué `makefile` usa `O_EXCL`?	Para que falle si el fichero ya existe y no machacarlo.

Pregunta	Respuesta corta
¿Por qué `delete` no puede borrar una carpeta con cosas?	Usa `rmdir`, que solo borra carpetas vacías. Para eso está `deltree`.
¿Por qué `deltree` se salta `.` y `..`?	`..` es la carpeta de arriba: si entrara, lo borraría todo hacia arriba. `.` es ella misma: bucle infinito.
¿Por qué `lstat` y no `stat`?	Con un enlace, `stat` mira a dónde apunta y `lstat` el enlace en sí. Así `deltree` no vacía la carpeta de destino y `list` no entra en bucles.
¿Qué es la recursividad en `deltree` y `list`?	Funciones que se llaman a sí mismas (o entre ellas) con cada subcarpeta, hasta que no quedan más.
¿Diferencia entre `-reca` y `-recb`?	Las dos entran en las subcarpetas; `-reca` lista la carpeta antes y `-recb`, después.
¿Qué pasa si un fichero no tiene dueño?	`getpwuid` devuelve `NULL`; usarlo daría segmentation fault, así que escribimos `??????`.
¿Por qué hay que poner el `\0` después de `readlink`?	`readlink` no lo pone, y sin él `printf` seguiría leyendo memoria.

Cómo compilar y probar

Cada vez que se cambia algo: compilar sin avisos, probar el comando y comprobar memoria.

```
cd ~/SistemasOperativos
gcc -Wall -Wextra p1.c -o p1     # no debe salir ningún aviso
./p1                             # ejecutar el shell
valgrind ./p1                    # al salir: "no leaks are possible" y "0
errors"
lsof -p <pid>                    # en otra terminal: comparar con listopen
```

- Un fallo en ejecución (segmentation fault) es un 0, así que hay que probar también entradas raras: líneas vacías, `close patata`, `readstr 3 -2`, `chdir noexiste`.
- Si hay muchos errores en la misma línea al compilar, casi siempre es un paréntesis o un `;`.

- Ante la duda de cómo debe comportarse un comando, se prueba en el shell de referencia de los profesores.
- Antes de seguir con cada grupo, se guarda una copia: `cp p1.c p1_grupo2.c`.

Cómo entregar

El zip se llama `p1.zip` (p minúscula) y dentro va **solo** la carpeta `P1` (P mayúscula) con `p1.c`. Como todo está en un fichero, no hace falta Makefile. Nada de ejecutables, `.o`, `.vscode`, `.git` ni ficheros de prueba. Lo entrega solo uno de los dos.

```
cd ~/SistemasOperativos
mkdir -p entrega/P1
cp p1.c entrega/P1/
cd entrega
zip -r p1.zip P1
unzip -l p1.zip                  # debe salir solo P1/ y P1/p1.c
```

Se hace desde la terminal y no desde el explorador de archivos, que puede meter ficheros ocultos. Mejor con margen: la entrega es el viernes 9 a las 23:00.